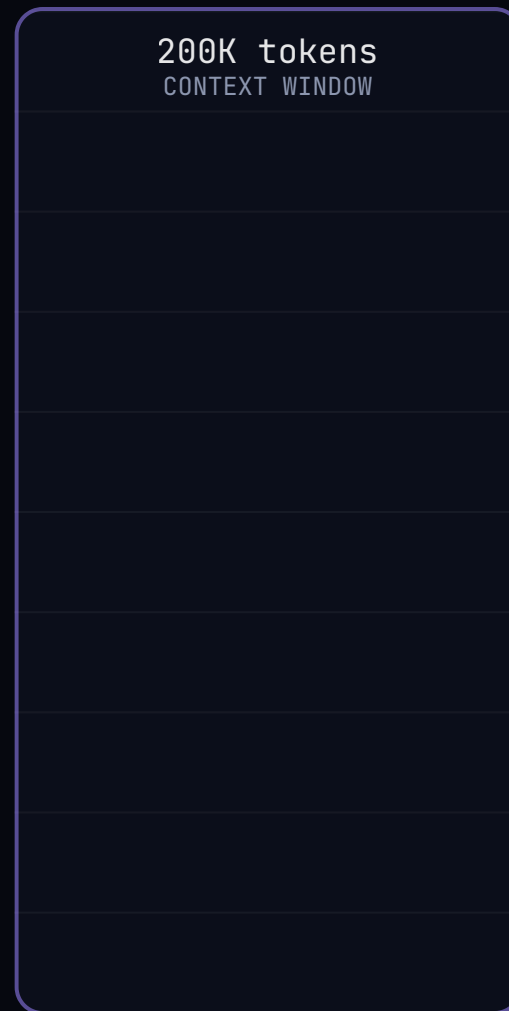


컨텍스트 — AI 에이전트의 작업 책상

에이전트의 머릿속엔, *딱 이 상자 하나*뿐입니다.

무엇을 올려두고 무엇을 치울지가 결과를 좌우합니다.

← 바이브 코딩이란 무엇인가? (원본 세미나).



agent의 작업 책상

AI는 이 상자 **밖**을 볼 수 없습니다

– 방향키로 진행하세요 ▶

📁 지난 세션
× 보이지 않음

📁 안 읽은 파일
× 보이지 않음

CONTEXT WINDOW

📐 설계서

💻 현재 코드

🔥 에러 로그

📄 오래된 문서

💬 무관한 대화

📖 중복 로그

🧠 당신의 의도
× 보이지 않음

🌐 회사 위키
× 보이지 않음

❌ **잡음이 섞이면 — 최고의 모델도 헛갈립니다**

상자에 없는 정보 = 세상에 없는 정보

크기는 정해져 있습니다

200K

📖 책 2~3권

소설 500쪽 분량의 텍스트가 통째로 들어가는 크기

📄 코드 2~3만 줄

웬만한 중형 프로젝트의 핵심 소스 전체

⚠️ 그런데 금방 찹니다

에이전트 세션에서는 파일·로그·대화가 쏟아져 들어와 생각보다 훨씬 빨리 소진됩니다

EN Fix the login bug and run the tests.

9 tok

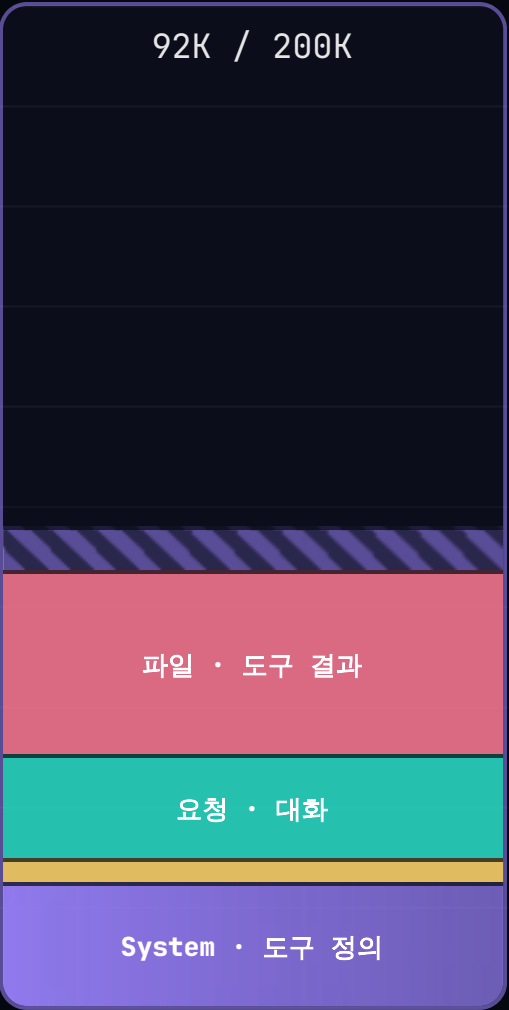
KO 로그인 버그를 고치고 테스트를 실행해줘.

~15 tok

같은 의미라도 **한국어는 영어의 ~1.5-2배** — 같은 200K여도 언어에 따라 체감 용량이 달라집니다.
그래서 긴 공통 지침은 영어로 쓰기도 합니다.

※ 이 발표의 모든 숫자는 감을 잡기 위한 **대략치**입니다. 모델·도구마다 다릅니다.

상자 안에는 무엇이 들어가나

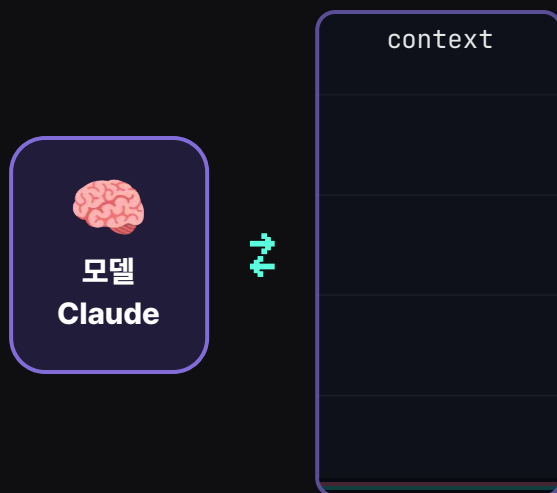


- System/Developer 지시 · 도구 정의 (도구가 자동 주입 - 다음 장) ~24K
- 메모리 · 프로젝트 지침 (CLAUDE.md · AGENTS.md · SKILL.md) ~4K
- User 요청 + 대화 히스토리 ~20K
- 검색 · 파일 · 도구 결과 (코드 · 로그 · 테스트) ~36K
- 에이전트의 중간 계획 (TODO · 판단) ~8K
- 여유 공간 — 남은 작업 예산 ~108K

에이전트는 질문에 답하는 모델이 아니라, 이 context를 재료로 다음 행동을 결정하는 시스템입니다.

맨 아래 'System'은 누가 쓰나 — 하네스

예: Claude Code = 하네스



'Claude Code'를 설치하면 — 모델 Claude를 감싼 이 프로그램이 깔리는 것

같은 구조 ▶

Codex × GPT

Antigravity CLI × Gemini

opencode × 선택 모델

설치해 쓰는 것은 모델이 아니라 모델을 감싼 프로그램 — 하네스. 하네스가 일하는 모습 ▾

🔗 System 프롬프트 · 도구 정의를 자동 주입

주입

🔧 모델이 요청한 도구를 실행하고 결과를 추가

실행

📄 상자가 차면 자동 압축 — 요약으로 교체

압축

같은 모델 + 다른 하네스 = 다른 에이전트

모든 정보의 무게는 같지 않다

1	시스템 규칙	절대 기준
2	개발자 · 도구 · 보안 규칙	강한 제약
3	프로젝트 지침	CLAUDE.md 등
4	사용자 요청	지금 할 일
5	현재 작업 관련 파일 · 문서	근거
6	실행 결과 · 로그	증거
7	이전 대화 요약	배경

충돌하면 어떻게 될까?

지침 "테스트 없는 수정은 하지 말 것"

요청 "그냥 빨리 고쳐줘"

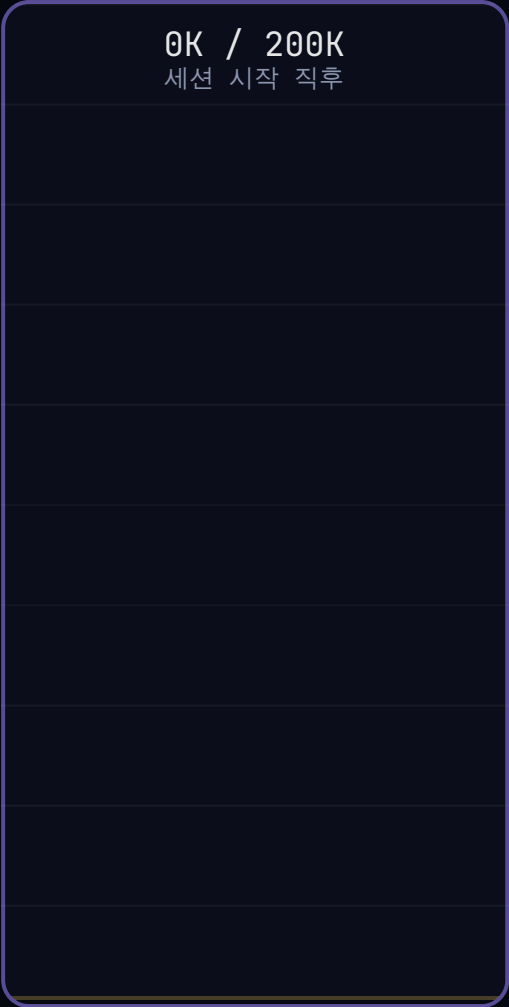
→ 좋은 에이전트는 지침을 따라 "수정 후 테스트까지" 판단합니다.

남은 정보가 남아 있으면

예전 결정 : SQLite 사용 · 현재 결정 : DuckDB 사용

둘 다 상자에 남아 있으면 에이전트는 어느 쪽을 따를지 흔들립니다.

첫 인사를 하기도 전에, 이미 ~15% - 방향키로 진행하세요 ▶



 세션 시작 — 상자는 비어 있음	0K
 시스템 프롬프트 주입	+3K
 도구 정의 30~40개 — 이름·설명·파라미터	+21K
 CLAUDE.md · 메모리 — 매 세션 자동 로드	+3K
상주 27K	실제 작업 예산 ~170K

여기 넣은 글자는 매 턴 자리를 차지하는 상주 비용입니다.

에이전트 루프 — 상자는 점진적으로 찬다 - 방향키로 진행하세요 ▶



세션 시작

27K

↺ 1바퀴 — 관련 파일 읽기

+9K

↻ 2바퀴 — 테스트 실행 로그

+15K

↻ 3바퀴 — 수정 diff · 재검증

+12K

모든 파일을 미리 읽지 않고, 필요한 것만 점진적으로
끌어옵니다.

[illegible]

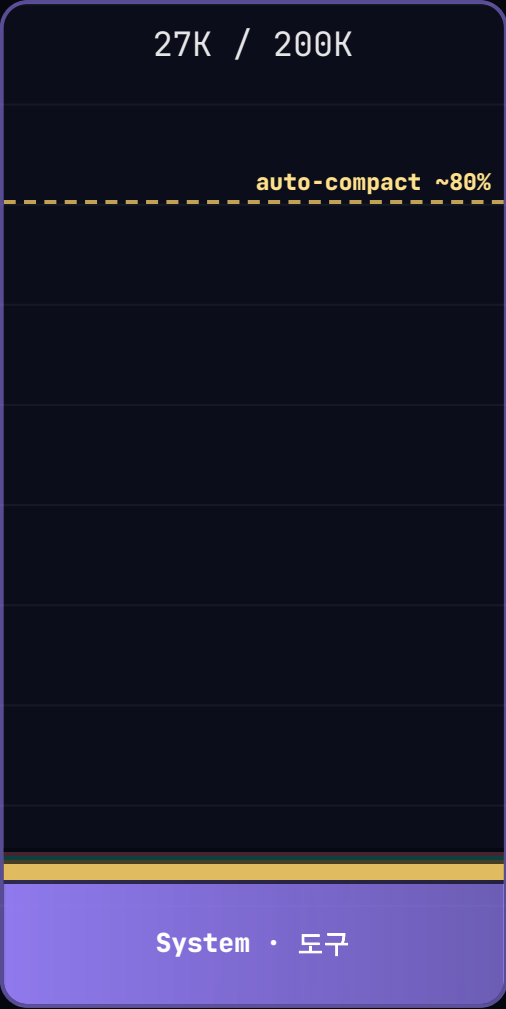
루프마다 조금씩 ↑

한 세션의 일생

— 방향키로 진행하세요 ▶

💬 세션 시작 — 시스템 + 도구 + CLAUDE.md	27K
👤 "로그인 버그 고쳐줘" + 계획 수립	+4K
📁 auth.py · login_service.py · test_auth.py 읽기	+35K
🖋️ 원인 추정 · 코드 수정 · diff 출력	+18K
🔧 테스트 실행 — 장황한 로그가 통째로	+40K
🔄 재수정 왕복 ×6 + 중간 판단	+42K

83% — 버그 하나 고쳤을 뿐인데.
"필요한 정보"와 "이제는 필요 없는 정보"가 뒤섞이기 시작합니다.



길어진다고 똑똑해지지 않는다

— 방향키로 진행하세요 ▶

🔑 "토큰 만료는 UTC 기준으로 계산할 것"

↑ **lost in the middle** — 중간의 핵심부터 묻습니다

🎯 주의 분산 — 잡음에 끌린다

🐸 낮은 전제 유지 — SQLite? DuckDB?

🔍 느려지고 비싸진다

📊 판단이 흐려진다

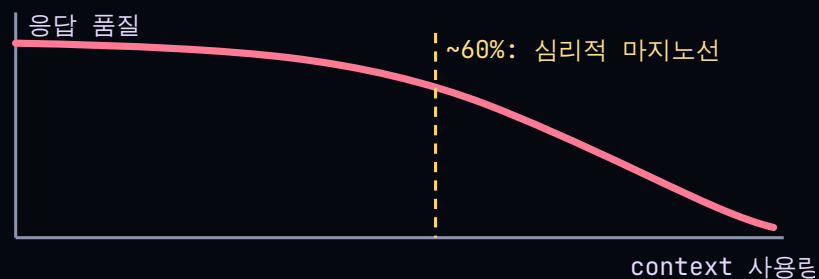
🔄 같은 파일을 또 읽는다

많이가 아니라, **지금 필요한 것만.**

가득 차면: 압축과 망각



한계선을 넘으면 자동 압축 — 과거 대화가 **요약본으로 교체**되고, 세부 결정·이유·예외가 사라집니다.



가득 차기 **전에** 이미 나빠집니다 — context rot

92% — 한계 임박

압축의 실체 — zip이 아니라 "요약해서 갈아끼우기"

LLM이 쓴 요약문으로 원문 히스토리를 통째로 교체합니다 — zip과 달리 복원 불가.

BEFORE — 원문 히스토리 141K

 "로그인 버그 고쳐줘"

 auth.py 읽기 — 812줄 전체

36K

 pytest 로그 3,200줄 · 실패 2건

40K

 원인: exp를 로컬 시간으로 계산해
UTC 서버와 1시간 어긋남

 "refresh token 로직은 건드리지 마"

 수정 diff · 재테스트 통과 ...

AFTER — 모델이 새로 쓴 요약 6K

"로그인 버그를 수정했다.
원인은 토큰 만료 시간 계산 오류였고,
auth.py를 고쳐 테스트가 통과했다."

요약에서 사라진 것 ↓

- UTC vs 로컬 — 구체 원인
- "refresh token 금지" 제약
- 코드 — 로그 원문

요약은 복원 불가이고, 무엇을 남길지는 요약 모델의 추측입니다.
그래서 /compact "auth 버그 수정 내용 위주로 남겨줘"처럼 남길 것을 직접 지시할 수 있습니다.

에이전트 성능은 곱셈이다

모델 능력
0.95

×

context 품질
0.30

×

도구 품질
0.90

×

검증 루프
0.85

=

체감 성능
0.22

context 품질이 낮으면, 최고의 모델도 이상하게 행동합니다.

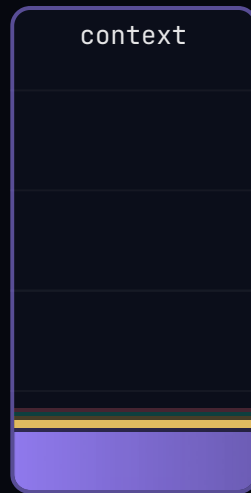
❌ 에이전트에게 불리한 요청
"이거 고쳐줘. 이전에 말한 방식으로."

'이전 방식'이 상자에서 밀려났다면
에이전트는 그것을 모릅니다

✅ 핵심 context가 명확한 요청
"이 버그를 고쳐줘. 단, 전역 상태는 추가하지 말고
auth_service.py 안에서 처리해줘.
수정 후 test_auth.py를 실행해서 확인해줘."

넣을 것 vs 뺄 것 - 방향키로 분류하세요 ▶

✓ 상자로



🎯 목표 · 완료 기준

🚫 제약 · 금지 사항

🔥 에러 로그 tail 20줄

이번 작업의 재료들

✗ 치운다



🗑 전체 로그 — tail이면 충분

🗑 폐기된 요구사항

🗑 참고용 문서 뭉치

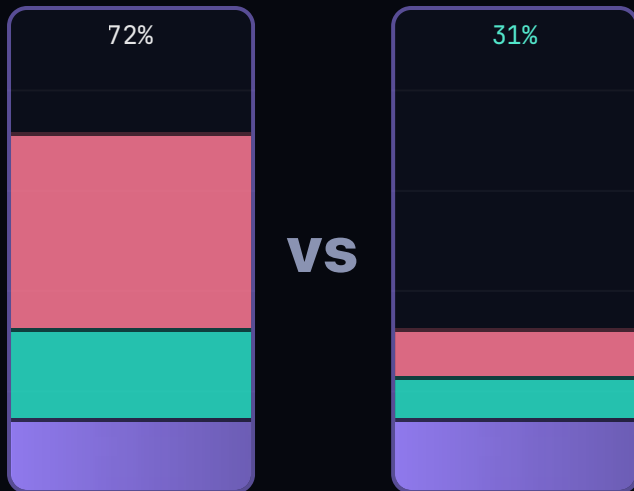
모든 것이 아니라, 이번 판단에 필요한 것만.

목표와 완료 기준을 분명히

❌ 나쁜 예
"이 코드 개선해줘."

무엇이 '개선'인지 에이전트가 탐색으로
알아내야 함 → 탐색 20~60K 토큰

✅ 좋은 예
"이 함수의 속도를 개선해줘.
동작은 바꾸지 말고, 기존 테스트는 모두 통과해야 해.
가능하면 복잡한 의존성 추가 없이 수정해줘."



파일과 함수를 **꼭 집어주면** 에이전트의 탐색 비용이 사라집니다.
"찾아서 고쳐줘" 대신 "src/auth/login.ts의 validateToken()을 고쳐줘"

긴 로그는 핵심만

❌ 나쁜 예

빌드 로그 30,000줄 전체 붙여넣기

한 번에 상자의 20~25%가 소음으로 채워짐

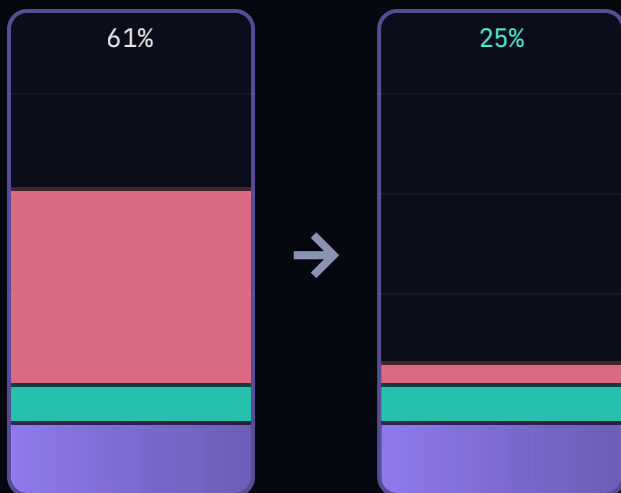
정작 중요한 한 줄은 중간에 묻힘

✅ 좋은 예

"마지막 에러는 아래와 같아.

```
ImportError: cannot import name 'X'  
    from 'module_y'
```

관련 변경은 `module_y.py`에서
함수명을 변경한 뒤 발생했어."



같은 정보량이라도 에러의 마지막 부분 + 맥락 한 줄이면 충분합니다. 로그는 요약본이나 tail만 전달하세요.

반복 지침은 파일로

매번 프롬프트에 쓰는 대신, 프로젝트에 두면 에이전트가 매 작업마다 같은 기준을 참고합니다.

AGENTS.md / CLAUDE.md / SKILL.md

- └ 코딩 스타일 · 컨벤션
- └ 테스트 방법
- └ 금지 사항 · 보안 규칙
- └ 폴더 구조
- └ 배포 방식

⚠ 단, 간결하게

이 파일은 매 세션 자동 로드되는 상주 비용 — 이력서처럼 쓰세요. 짧고, 늘 참인 것만.

기억은 **파일로** — 상자는 휘발성, 파일은 영속

컨텍스트는 RAM, 파일은 **디스크**입니다.

💬 말로만 남긴 기억

대화 "포트는 8081을 쓰기로 했어"

→ 30턴 뒤 자동 압축 → 요약에서 누락 →
에이전트가 다시 8080으로 되돌립니다

📝 파일로 남긴 기억

"이번에 정한 규칙과 이유를
`docs/decisions.md`에 기록해줘"
—— `/clear` · 다음 날 새 세션 ——
"`docs/decisions.md` 읽고 이어가줘"

압축 · 초기화 후에도 **그대로 복구**

📋 PLAN.md · TODO

긴 작업의 체크리스트를 파일로 두고 단계마다 갱
신 — "어디까지 했나"가 압축을 건드립니다

📄 NOTES.md

대량 조사 결과·중간 발견을 노트 파일로.
Claude가 공략 노트를 쓰며 포켓몬을 플레이한
것과 같은 원리

🗄️ MEMORY.md · memory tool

하네스가 기본 지원하는 파일 기반 장기 기억 —
Claude Code의 auto memory, API의
memory tool

길어지면 요약하고, 비우고

📝 중간 요약

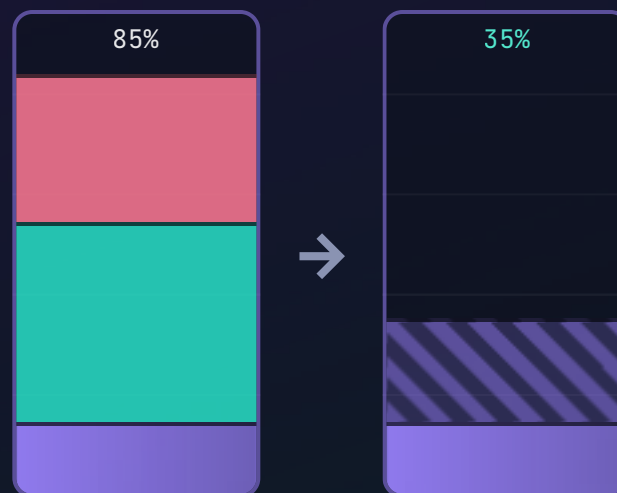
현재까지 결정된 것:

- 인증 로직은 `auth_service.py`
- DB 스키마는 변경하지 않음
- 원인: token 만료 계산 오류

다음 단계: `test_auth_expiry` 실행

긴 대화 전체보다 훨씬 유용한 압축본

/compact



이어가야 할 때 — 대화를 요약본으로 교체

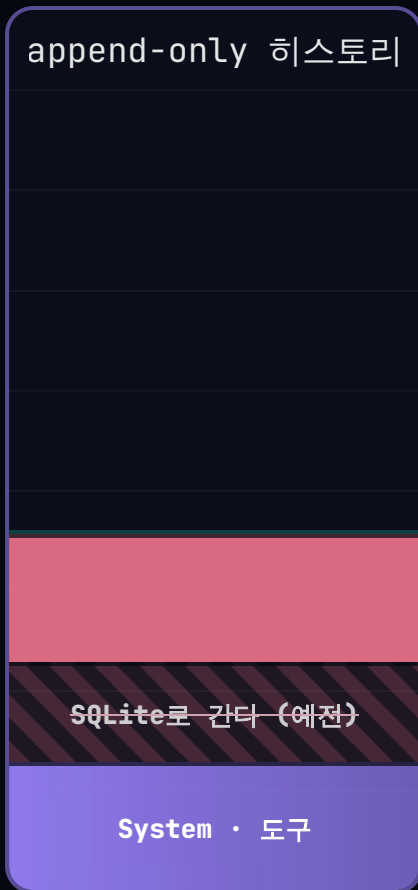
/clear · 새 세션



작업 하나 = 세션 하나가 기본기

이어갈 이유가 없다면, 비우는 게 항상 이득입니다.

폐기 선언은 **지우개가 아니라 덮어쓰기**.



대화 히스토리는 덧붙기만(append-only) 합니다 — 폐기를 선언해도 옛 문장은 남습니다.

 그래도 선언하는 이유 — 최신·명시적 신호가 이깁니다
"이전의 SQLite 기반 접근은 폐기하고,
이제부터는 DuckDB + Parquet 기준으로 진행해줘."
옛 결정보다 더 최근 위치에, 더 명시적인 상위 지시가
추가되어 모델이 이를 따르게 됩니다

진짜로 상자에서 치우려면

- ① /clear · 새 세션 — 유효한 결정은 파일에 남기고
- ② /compact "SQLite 논의는 버려줘" — 요약에 반영
- ③ API의 context editing — 오래된 도구 결과를 자동 삭제

옛 문장은 그대로 남습니다

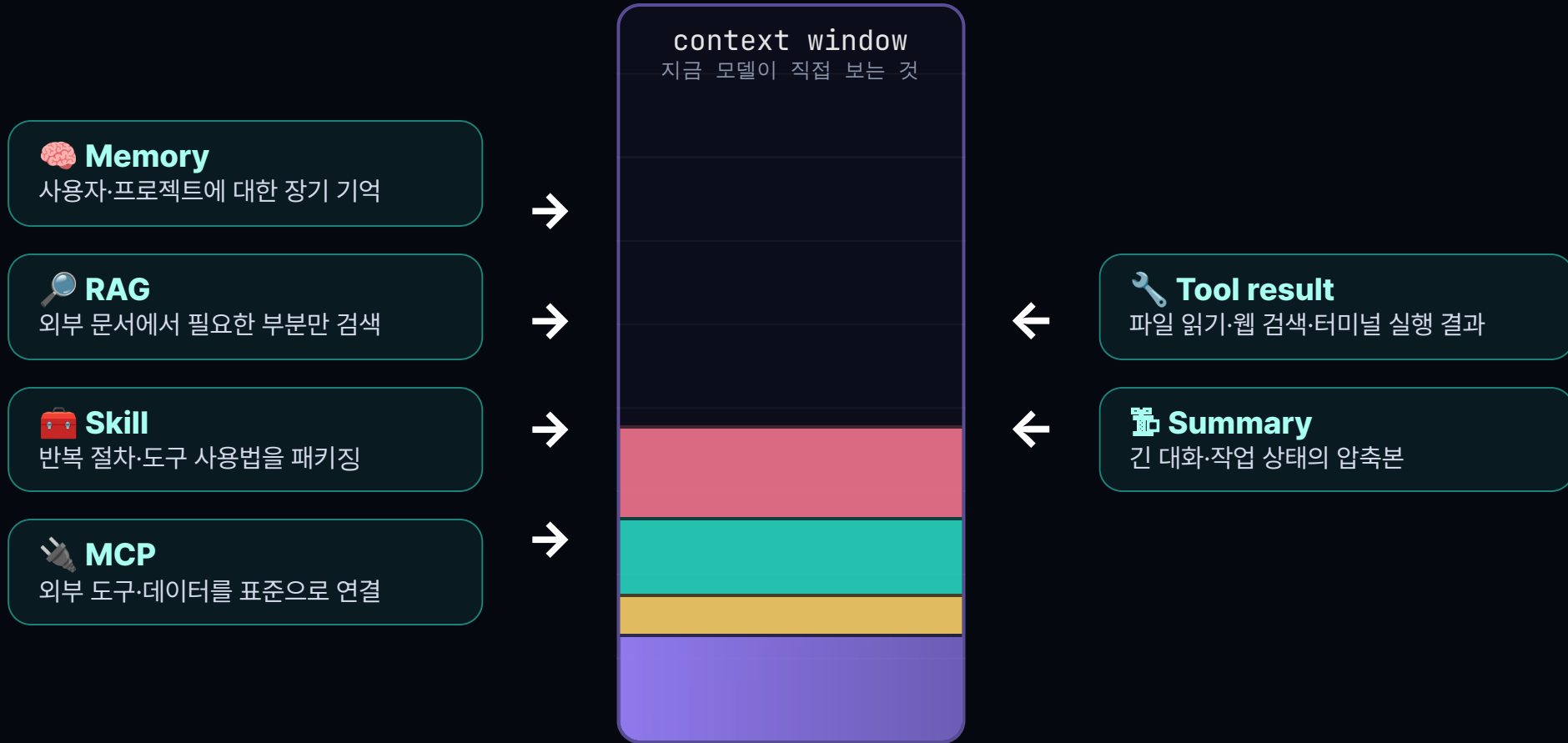
서브에이전트 = 컨텍스트 격리



서브에이전트는 자기만의 상자를 새로 받습니다. 대량 탐색·파일 읽기 같은 지저분한 소비는 그 안에서 끝나고, 본진 상자에는 결론 몇 K만 착륙합니다.

코드에서 함수를 분리하는 것과 같은 원리입니다. → [심화 세미나: 서브에이전트와 오케스트레이션](#)

상자 밖의 세계 — 필요할 때만 안으로



Memory·RAG·Skill·MCP는 상자 밖에 있다가, 필요할 때 상자 안으로 들어와야 실제 판단에 쓰입니다.
문서가 아무리 많아도, 검색되어 들어오지 않으면 없는 것과 같습니다.

MCP — 도구를 끗는 표준 포트 - 방향키로 서버를 끗아보세요 ▶

Model Context Protocol — 하네스에 외부 도구·데이터 소스(GitHub·Slack·사내 DB...)를 연결하는 공개 표준. "AI 도구의 USB-C 포트"라고 불립니다.

🔧 기본 도구만 — 파일 · 터미널 · 검색

24K

🔗 + GitHub 서버 — 도구 35개의 이름·설명·파라미터

+18K

🗨️ + Slack 서버 — 도구 11개

+12K

+ Jira · 사내 DB 서버 ...

+18K

첫 인사도 전에 36% — 도구의 정의도 전부 컨텍스트입니다.

그래서 최신 하네스는 도구 이름만 올려두고 상세 스키마는 필요할 때 검색해 로드합니다(Tool Search). 안 쓰는 서버는 빼는 게 기본입니다.

24K / 200K

대화 시작 전

System · 도구 정의

Agent Skill — 절차를 파일로 패키징

— 방향키로 진행하세요 ▶

SKILL.md 전체 — deploy-check 스킬

✓ 항상 context 안

```
---
name: deploy-check
description: 배포 전 점검 절차.
  배포·릴리스 요청 시 사용
---
```

YAML frontmatter — 이것만 항상 상주 · ~50 tok

→ context 로드됨

```
# 절차 (본문 — 아무리 길어도 OK)
1. 테스트 전체 실행 · 커버리지 확인
2. CHANGELOG 갱신 · 버전 태그
3. 스테이징 배포 → 스모크 테스트
... (500줄)
```

본문 · ~2K — 관련 작업이 감지된 순간에만

→ context 로드됨

```
└─ reference.md   ← 37K짜리 상세 규칙
└─ scripts/check.sh ← 실행만, 코드는 상자 밖
```

부속 파일 · +α — 그 단계에 도달했을 때만

PROGRESSIVE DISCLOSURE — 단계적 공개 ▶

- 1 평소: 모든 스킬의 frontmatter만 상주 ~50 tok/스킬
- 2 관련 작업 감지 → 그 SKILL.md 본문만 로드 ~2K
- 3 필요할 때만 부속 파일 읽기 · 스크립트 실행 +α

매뉴얼을 통째로 외우게 하지 않고,
목차만 들고 있다가 필요한 장을 펴게 한다.

본문이 길어도, 부속 파일이 많아도 — 평소 비용은 frontmatter 몇 줄입니다.

같은 도구, 두 가지 연결법 — MCP vs CLI + Skill

과제: "열린 PR을 확인하고 리뷰 코멘트를 남겨줘" — GitHub에 연결하는 두 가지 방법

MCP 서버 직결

~18K
상주

도구 35개의 이름·설명·스키마가 매 세션 로드 — 그날 안 써도

호출 결과(JSON)도 매번 상자를 통과

도구 정의

CLI 도구 + Skill

~50 tok
상주

상주는 **frontmatter** 몇 줄뿐

필요할 때: SKILL.md ~2K 로드 → `gh pr list` 실행.

모델은 이미 `git` · `gh` 사용법에 익숙

쓰지 않는 날의 상주 비용: 18K vs 0.05K — 약 1/300

단, 만능 공식은 아닙니다 — MCP가 정답일 때

 원격 서비스 + OAuth 인증 ·  셸·CLI가 없는 실행 환경(웹 챗 등) ·  다중 사용자 권한·감사 로그가 필요한 제품

MCP도 진화 중 — Tool Search(스키마를 필요할 때만 로드) · 코드 실행 결합으로 150K→2K(−98.7%)

사내 환경: 효율을 넘어 **보안**

상자에 담는 것 = 외부 모델로 나가는 것

사내 Agent에는

내부 코드 · 내부 문서 · 사내 로그 · 보안이 필요한 데이터

 원본 그대로

"Pixel ABC 프로젝트의 장비 X에서
wafer Y 수율이 72.3%로 떨어졌고..."

사외 Agent에는

일반 개념 설명 · 공개 문서 · 샘플 코드 · 민감 정보를 제거해 추상화한 문제

 재구성된 context

"어떤 제조 데이터 분석 파이프라인에서
특정 배치의 품질 지표가 급락했을 때,
원인 분석용 Python 구조를 어떻게 설계하면 좋을까?"

사외 Agent에는 "원본 context"가 아니라 민감 정보를 제거한 재구성 context를 주세요.

→ 심화 세미나: 사내 환경에서의 AI — 보안과 거버넌스

REFERENCES

더 깊이 보기

공식 문서

[Explore the context window](#)

Claude Code Docs — 상자가 차는 과정 인터랙티브 시뮬레이션

[How Claude remembers your project](#)

CLAUDE.md · auto memory

[Model Context Protocol](#)

MCP 공식 사이트 · 스펙과 튜토리얼

[Context editing · Memory tool](#)

Claude Platform Docs — API 레벨 컨텍스트 관리

엔지니어링 블로그

[Effective context engineering for AI agents](#)

Anthropic — 이 발표의 뼈대

[Equipping agents with Agent Skills](#)

Anthropic — progressive disclosure

[How we built our multi-agent research system](#)

Anthropic — 서버에이전트 컨텍스트 격리

[Context Engineering: Lessons from Manus](#)

Manus — "파일 시스템이 궁극의 컨텍스트"

연구 · 규약

[Context Rot](#)

Chroma, 2025 — 입력이 길수록 성능이 흔들린다

[Lost in the Middle](#)

Liu et al., TACL 2024 — 중간 정보를 놓친다

[AGENTS.md](#)

에이전트 지침 파일의 공통 규약

[Claude Code best practices](#)

Anthropic — 에이전틱 코딩 실전 팁

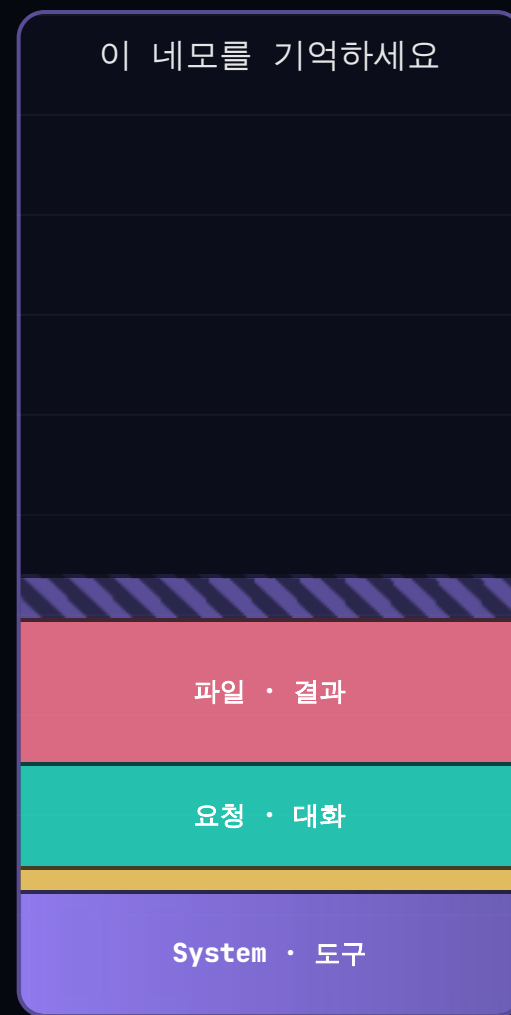
CLOSING

좋은 사용자는 context 설계자입니다

1. 상자 밖은 존재하지 않는다
2. 상자는 유한하고, 시작부터 ~15%는 없다
3. 모든 행동에는 토큰 가격이 있다
4. 60%를 넘기 전에 — 비우거나, 쪼개거나, 격리하라

좋은 답을 **요구**하는 것이 아니라,
좋은 판단을 할 수 있는 **작업 환경**을 만들어주는 것.

← 원본 세미나: 바이브 코딩이란 무엇인가?



한정된 공간에 무엇을 올리고 무엇을 치울지 —
그 결정이 에이전트 활용의 품질을 좌우합니다.